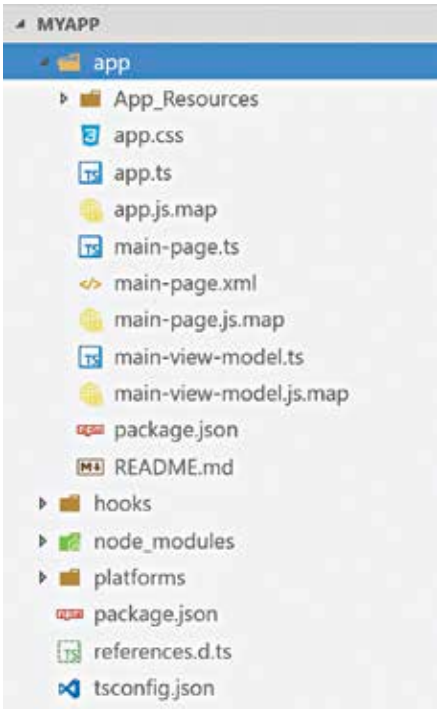
I said earlier, VS Code is just a text based editor and does a fantastic job in editing files like JS/CSS/XML and many other file types. Here is a snapshot of the new project we have created:



NativeScript Project Structure

Let's go over the different pieces in the project:

- **appfolder** – This is where the application related code resides. This is where you add new screens, business logic, style sheet etc. You will be working mostly in this folder.
- **App_Resources folder** –It contains your app assets such as icons, splash screens etc.
- **node_modules folder** – This is a folder where all the dependencies from npm are downloaded.
- **platforms folder**–Whenever we add a target platform, NativeScript will copy certain core platform related dependencies such as build scripts etc. to this folder
- **Package.json**– this file is used for listing down your projects node module dependencies.
- **tsconfig.json** – Since I am using TypeScript as the language, this is a configuration for TypeScript compiler on how to convert my TypeScript code to JavaScript.

Bootstrapping the App:

Most of the things in NativeScript are guided by convention. NativeScript runtime will look for app.js file in your app. Note: During development time the file extension is .ts but when compiled, TypeScript compiler will generate .js file. Basically, TypeScript being a superset cannot be executed directly. It needs to be converted to ECMA Script Version 5. As a developer, I do not have to worry about that. The NativeScript compilation will take care of that. Just remember that your app starting point at runtime is app.js. Let's take a look at the code and understand what is happening:

```
import * as app from 'application';
app.start({ moduleName: 'main-page' });
```

We import a core module called application. Then we say what screen/page of our app needs to be the starting page of our app. Notice we just say "main-page" – no extension provided. That is because, as I said earlier, NativeScript works by convention rather than configuration. NativeScript will now look for any page/screen with name main-page.xml and open that page after launching the app. As simple as that.

XML for UI Definition:

In NativeScript we define application screens/pages using XML syntax. XML as a language is great for defining hierarchical structure. Hence in NativeScript we do not reinvent the wheel but rather stick to what every developer knows. Here is the code snippet of main-page.xml:

```
<Page xmlns="http://schemas.nativescript.org/tns.xsd" navigatingTo="navigatingTo">
<StackLayout>
<Label text="Tap the button" class="title"/>
<Button text="TAP" tap="{{ onTap }}" />
<Label text="{{ message }}"
class="message" textWrap="true"/>
</StackLayout>
</Page>
```

At the root, we have a Page element and it has some child elements. Page itself is a cross-platform NativeScript module. It knows how to render itself natively on different platform. Then we have a StackLayout – a layout element which stacks its child elements horizontally or vertically. StackLayout contains a label with text "Tap the Button", a button with text "Tap" and another label to display a message. Page exposes certain lifecycle events and we have shown interest in one which has an

*pointers

>>Interesting Django and other developer videos

*NativeScript 2.0

>>The launch webinar of NativeScript 2.0 that talks in detail about what it brings to the table.

http://dgit.in/NtvScpt

*Visual Studio Code

>>Gavin Bauman talks about new features rolled into Visual Studio Code, Microsoft's free and cross-platform editor.

http://dgit.in/VSCodeNS

*Unit Testing in NS

>>How to setup and use the built in unit testing in NativeScript to test app code as well as plugins.

http://dgit.in/UnitTst

*Resilient Apps with Angular 2

>>Build resilient production ready apps with Angular 2, TypeScript, and Redux that are offline ready.

http://dgit.in/NScptRA